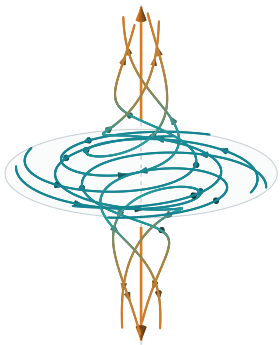
What you need to know about Lean

Yoh Tanimoto

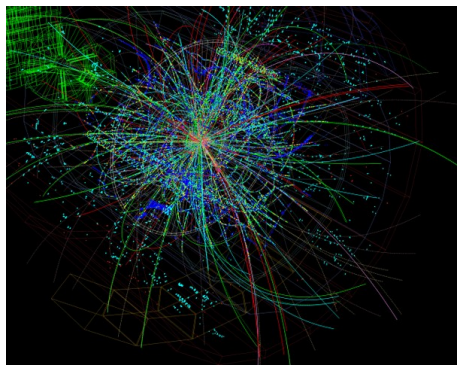
AI per Matematica e viceversa
11 November 2026

AI proving and verifying theorems

- Erdős Problems (database, Lean statements)
- Complex structure on S^6 (paper, Lean)
- Navier–Stokes blowup with forcing (paper, Lean)
- 721 manuscripts (manuscripts, Lean (∼42% of main results formalized))
- Yang–Mills existence? (announcement, paper not yet, Lean)
- Fermat's Last Theorem, verification (announcement, Lean)
- Classification of finite simple groups, verification (website, Lean)



Blowup core. OpenAI, Finite time blowup for Navier–Stokes, Fig. 1



Proton–lead collision in ALICE (CERN), 2012. Pcharito, CC BY-SA 3.0

Lean certificate

Some say the proofs are correct because they come with a “Lean certificate”. What is it?

```
14
15    /-- (C) Breakdown of Navier-Stokes solutions on ℝ³. -/
16    theorem navier_stokes_breakdown_R3 (nu : ℝ) (hnu : nu > 0) :
17        ∃ (u₀ : ℝ³ → ℝ³) (f : ℝ³ → ℝ → ℝ³),
18        InitialVelocityConditionDecay u₀ ∧ ForceConditionDecay f ∧
19        ¬ (∃ v p, NavierStokesExistenceAndSmoothnessRn nu u₀ f v p) := by
20      exact ComparatorBridge.navier_stokes_breakdown_R3 nu hnu
21
22    /-- (D) Breakdown of Navier-Stokes solutions on ℝ³/ℤ³. -/
23    theorem navier_stokes_breakdown_periodic (nu : ℝ) (hnu : nu > 0) :
24        ∃ (u₀ : ℝ³ → ℝ³) (f : ℝ³ → ℝ → ℝ³),
25        InitialVelocityConditionPeriodic u₀ ∧ ForceConditionPeriodic f ∧
26        ¬ (∃ v p, NavierStokesExistenceAndSmoothnessPeriodic nu u₀ f v p) := by
27      exact ComparatorBridge.navier_stokes_breakdown_periodic nu hnu
```

(NavierStokes/ComparatorSolution.lean)

Lean, an interactive theorem prover

In Lean,

- you can write definitions and theorem statements
- you can write the proofs
- **Lean checks the proofs**

```
def LinearlyIndependent (x y : ℝ × ℝ) : Prop :=
  ∀ a b : ℝ, a • x + b • y = 0 → a = 0 ∧ b = 0
```

The vectors $(1, 0)$ and $(1, 1)$ in $\mathbb{R}^2$ are linearly independent.

```
example : LinearlyIndependent (⟨1, 0⟩ : ℝ × ℝ) (⟨1, 1⟩ : ℝ × ℝ) := by
  intro a b h
  have h' : ((a + b, b) : ℝ × ℝ) = (0, 0) :=
    calc ((a + b, b) : ℝ × ℝ) = a • (1, 0) + b • (1, 1) := by simp
      _ = 0 := h
  have hab : a + b = 0 := (Prod.ext_iff.mp h').1
  have hb : b = 0 := (Prod.ext_iff.mp h').2
  have ha : a = 0 := by linarith
  exact ⟨ha, hb⟩
```

What is a mathematical proof?

Natural deduction based on inference rules:

	$\wedge$	$\vee$	$\to$
intro	$\dfrac{P \quad Q}{P \wedge Q}$	$\dfrac{P}{P \vee Q} \quad \dfrac{Q}{P \vee Q}$	$\dfrac{\begin{array}{c}[P] \\ \vdots \\ Q\end{array}}{P \to Q}$
elim	$\dfrac{P \wedge Q}{P} \quad \dfrac{P \wedge Q}{Q}$	$\dfrac{P \vee Q \quad P \to R \quad Q \to R}{R}$	$\dfrac{P \quad P \to Q}{Q}$

Example: a proof of $P \wedge Q \to Q \wedge P$

$$\dfrac{\dfrac{\dfrac{[P \wedge Q]^1}{Q}\,\wedge\mathrm{E} \quad \dfrac{[P \wedge Q]^1}{P}\,\wedge\mathrm{E}}{Q \wedge P}\,\wedge\mathrm{I}}{P \wedge Q \to Q \wedge P}\,\to\mathrm{I}^1$$

What is a Lean proof?

Inference rules are declarations in Lean (`h : a` means `h` is a proof of `a`):

```
-- ∧ intro, elim
And.intro {a b : Prop} (left : a) (right : b) : a ∧ b
And.left {a b : Prop} (self : a ∧ b) : a
And.right {a b : Prop} (self : a ∧ b) : b
-- ∨ intro, elim
Or.inl {a b : Prop} (h : a) : a ∨ b
Or.inr {a b : Prop} (h : b) : a ∨ b
Or.elim {a b c : Prop} (h : a ∨ b) (left : a → c) (right : b → c) : c
-- → intro, elim (built into the language)
fun (h : a) => t : a → b -- if t : b, possibly using h
f h : b -- if f : a → b and h : a (application)
```

Example (link):

```
example (P Q : Prop) : P ∧ Q → Q ∧ P := by
  intro h
  exact And.intro (And.right h) (And.left h)
```

Lean's foundation

Lean uses **dependent type theory** as its basis. Everything has a **type**.

- `Nat : Type` ($= \mathbb{N}$)
- `zero : Nat` ($= 0$), `succ zero : Nat` ($= 1$).
- `Prop : Type` (propositions)
- `h : P ∧ Q` (`h` is a proof of `P ∧ Q`)
- $\mathbb{N} \to$ `Prop : Type` (predicates that depend on a natural number)
- `Type : Type 1`
- From types, we can build new types (pairs, functions, quotients, ...)

Different foundations, and interactive theorem provers based on them:

set theory (e.g. ZFC) in first-order logic	Metamath, Mizar
higher-order logic (= simple type theory)	Isabelle/HOL, HOL Light
dependent type theory	Rocq, Lean

Lean (and dependent type theory) is one choice.

How does the Lean kernel check a proof?

- A theorem statement, say `P`, has type **`Prop`**.
- Its proof has type `P`.
- After "**`example`** `: P :=`", you need to construct an object of type `P`.
- (you can use tactics such as **`intro`**, **`exact`**)
- There are only certain ways to construct such an object, corresponding to the inference rules.

- The Lean kernel checks whether the object you construct has type `P`.
- Only the (small) kernel needs to be trusted, not tactics or AI.
- (cf. Talk by F. van Doorn in 2024 at Tor Vergata)

Can we trust Lean kernel?

Soundness bugs (where Lean accepted a proof of `False`)

- Auxiliary types not checked in a very special case (#14576, postmortem)
- Wrong use of cache (#14806)
- Missing checks on opaque definitions, projections (#14484, #14807)
- Reference counter overflow (memory bug) (#14838)
- Bugs in an external library (GMP) (#14833)

The bugs listed here have been fixed.
Proofs can be re-checked by external type checkers (link).

Consistency: abstract theory and a verified checker

- **Abstract type theory:** Carneiro (2019) (gaps remain (Hales, 2026))
- **Verified checker:** con-leche (written in Lean) has a formal consistency proof: a proof checked by Lean that the checker cannot accept a proof of `False`, relative to ZFC + inaccessible cardinals.

Example: Analisi matematica I

If f and g are continuous on $\mathbb{R}$, then $f + g$ is continuous on $\mathbb{R}$.

```
def Continuity (f : ℝ → ℝ) : Prop :=
  ∀ x : ℝ, ∀ ε > 0, ∃ δ > 0, ∀ y : ℝ, |y - x| < δ → |f y - f x| < ε

example (f g : ℝ → ℝ) (hf : Continuity f) (hg : Continuity g) :
    Continuity (fun x => f x + g x) := by
  intro x ε hε
  obtain ⟨δ₁, hδ₁, h₁⟩ := hf x (ε / 2) (by linarith)
  obtain ⟨δ₂, hδ₂, h₂⟩ := hg x (ε / 2) (by linarith)
  refine ⟨min δ₁ δ₂, lt_min hδ₁ hδ₂, fun y hy => ?_⟩
  have e₁ := h₁ y (lt_of_lt_of_le hy (min_le_left _ _))
  have e₂ := h₂ y (lt_of_lt_of_le hy (min_le_right _ _))
  calc |f y + g y - (f x + g x)|
      = |(f y - f x) + (g y - g x)| := by ring_nf
    _ ≤ |f y - f x| + |g y - g x| := abs_add_le _ _
    _ < ε / 2 + ε / 2 := add_lt_add e₁ e₂
    _ = ε := by ring
```

(link) AI can easily write such a proof. Ask in a coding agent.

Example: Fermat's last theorem

(Mathlib statement)

```
/-- Statement of Fermat's Last Theorem over a given semiring with a
    specific exponent. -/
def FermatLastTheoremWith (R : Type*) [Semiring R] (n : ℕ) : Prop :=
  ∀ a b c : R, a ≠ 0 → b ≠ 0 → c ≠ 0 → a ^ n + b ^ n ≠ c ^ n

/-- Statement of Fermat's Last Theorem over the naturals for a given
    exponent. -/
def FermatLastTheoremFor (n : ℕ) : Prop := FermatLastTheoremWith ℕ n

/-- Statement of Fermat's Last Theorem: `a ^ n + b ^ n = c ^ n` has no
    nontrivial natural solution
when `n ≥ 3`.
-/
def FermatLastTheorem : Prop := ∀ n ≥ 3, FermatLastTheoremFor n
```

A Lean proof has been constructed (GitHub repo by Anthropic)

mathlib

- Lean : the formal computer language
- mathlib : the mathematical library in Lean

mathlib contains algebraic and analytic structures on Nat, Int, Rat..., the definition of Real, Complex.
More abstract structures as well such as

- algebraic Mul, Semigroup, Monoid, Group, Module, Algebra
- topological TopologicalSpace, MetricSpace, CompleteSpace
- analytic integral, fderiv, InnerProductSpace, CStarAlgebra
- geometric Bundle.RiemannianMetric, ContMDiffVectorBundle
- and **many many more**
- but many research-level definitions are **missing**

mathlib statement of the Riemann hypothesis (link)

```
def RiemannHypothesis : Prop :=
  ∀ (s : ℂ) (_ : riemannZeta s = 0) (_ : ¬∃ n : ℕ, s = -2 * (n + 1))
    (_ : s ≠ 1), s.re = 1 / 2
```

Bad definitions

```
theorem Give_me_1M_USD : Riemann_Hypothesis := by
  apply my_brilliant_proof

#print axioms Give_me_1M_USD
```

(link)

Bad definitions

```
theorem Give_me_1M_USD : Riemann_Hypothesis := by
  apply my_brilliant_proof

#print axioms Give_me_1M_USD
```

(link)

- Here `Riemann_Hypothesis` is different from `mathlib`'s `RiemannHypothesis`.
- `Riemann_Hypothesis` is the trivial statement, $1 + 1 = 2$.
- `#print axioms` says "does not depend on any axioms".
- (`propext`, `Classical.choice` and `Quot.sound` are considered the canonical axioms)

Bad definitions

```
theorem Give_me_1M_USD : Riemann_Hypothesis := by
  apply my_brilliant_proof

#print axioms Give_me_1M_USD
```

(link)

- Here `Riemann_Hypothesis` is different from `mathlib`'s `RiemannHypothesis`.
- `Riemann_Hypothesis` is the trivial statement, $1 + 1 = 2$.
- `#print axioms` says "does not depend on any axioms".
- (`propext`, `Classical.choice` and `Quot.sound` are considered the canonical axioms)

Can we trust Lean formalization by AI?

- Lean checks **proofs**.
- Lean does not check **definitions** and **statements**.

Can we trust AI translation of LaTeX into Lean?

No (at least there is no mechanical check)

- (For well-known things, AI is not too bad)
- Lean checks **proofs**.
- Lean **does not check definitions and statements**.
- You should verify the definitions and statements.

Even when the statement is translated correctly, if you ask AI to translate a natural language (NL) proof into Lean,

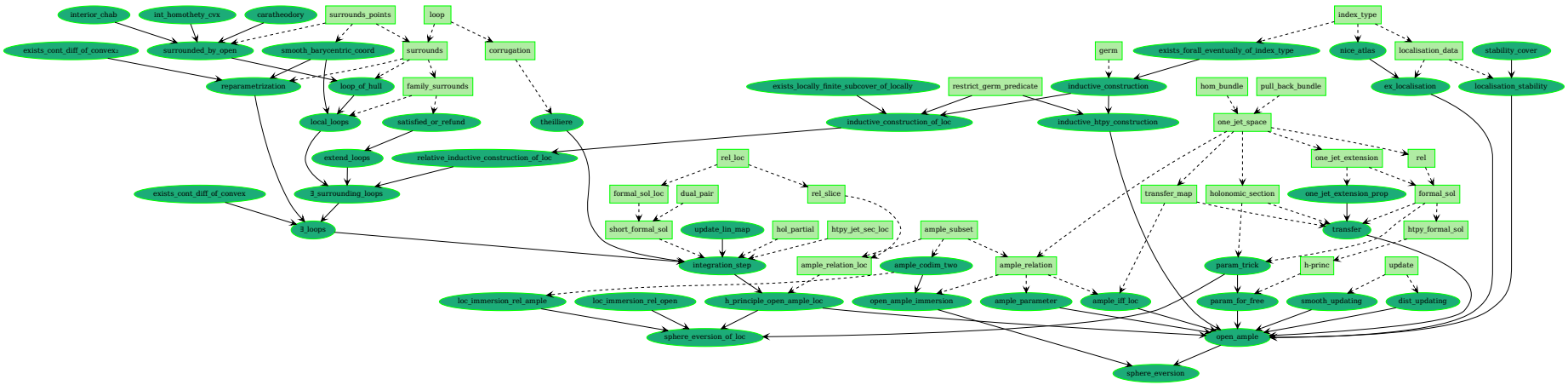
- AI may write a correct Lean proof without correcting the NL proof.
- Then the Lean code compiles and the Lean proof is correct, but the NL proof is still wrong.
- (No one reads the AI generated Lean proof)

cf. Bastounis, Circelli, Hansen, Navier–Stokes lost in translation (2026)

How can we write a Lean proof of my paper?

Tentative workflow:

- Put your paper in a folder with a Lean project.
- Identify the statement.
- Check whether you can state it using mathlib definitions.
- If not, check Tau Ceti and Formal Frontier.
- If not, let AI write the definitions and statements, then verify them.
- Write a blueprint (a very detailed informal proof linked with Lean code). Ask AI to follow the blueprint.



Blueprint dependency graph of The sphere eversion project (green: formalized)
P. Massot, O. Nash, F. van Doorn

How can we let AI write Lean proof of my paper?

(I haven't tried any of the following)

- Formalize your paper in Lean on Lean Machine Learning
  1. Get AI an agent (Claude Code, Codex...) with subscription
  2. Ask AI to install software
  3. Ask AI to write the statement
  4. Ask AI to write the proof
  5. Check the result (no `sorry`'s, no bad axioms), **check the definitions and statements**
- Autoformalization is now very easy by Scott Armstrong

How much are you willing to pay?

- €0 (**failed** with ChatGPT free): $a, b : \mathbb{R}, a \leq b, 0 \leq a \longrightarrow a^2 \leq b^2$
- $\sim$ €20/month (or open weight model): AM1 continuity exercise
- $\sim$ €100/month (or open weight model?): (your paper?)
- $\sim$ about €Million?: Fermat's last theorem

What if someone writes a Lean proof of my conjecture?

Tentative workflow:

Check whether the proof is complete:

- It builds, no `sorry`, `#print axioms` shows only standard axioms.

Check whether the definitions and statements are correct:

- Ask AI to identify the main statement, write it in NL, and find the definitions it depends on.
- If these definitions depend on other definitions, repeat recursively until you are only using trusted definitions (e.g. `mathlib`).
- Read the statement and these definitions yourself.

Try to understand the contents:

- Ask AI to identify the main intermediate steps and lemmas.
- Ask AI to write them in NL and a blueprint (or use LeanArchitect).
- Try to understand these small pieces and the large structure.

Is Lean useful outside math?

Software/hardware verification

Theorem provers have been mainly developed by computer scientists, with real applications to verification of hardware and software. "Mistakes can be very costly, examples are the destruction of the Ariane 5 rocket (caused by a simple integer overflow problem that could have been detected by a formal verification procedure) and the error in the floating point unit of the Pentium II [sic] processor." (Bridge, 2010. link)

"The failure resulted in a loss of more than US$370 million." (Wikipedia entry about Ariane flight V88 in 1996)

Lean was created in 2013 by Leonardo de Moura (then at Microsoft Research, now at AWS). Now it is used at AWS to verify Cedar, an authorization policy language, and in the (long-term) project Signal Shot to verify the messaging app Signal (ask Oliver!).

Libraries and tools

- Mathlib (human-curated library)
- Tau Ceti (human-directed, AI generated library)
- Formal Frontier (AI formalization of textbooks)
- Blueprint (informal proof in LaTeX linked to the Lean code, with a dependency graph)
- Comparator (checks that a proof proves exactly the given statement, with the same definitions and only the allowed axioms)
- External type checkers (if you don't trust Lean kernel)

Installation / making a new Lean project

Install:

- VS code
  - Lean extension
  - Coding agent (Claude Code, Codex,...), often under subscription
- Lean installation guide

Start a project depending on Mathlib (guide):

1. VS Code: $\forall \to$ New Project $\to$ Create Project Using Mathlib (it also downloads the compiled Mathlib)
2. In a new `.lean` file in the project, start with `import Mathlib`

Or ask the agent:

1. Open an empty folder in VS Code and start the agent
2. "Create a Lean project depending on Mathlib, download the Mathlib cache, and check that it builds."
3. Check that `lake build` succeeds

Learn

Lean portal : Lean Programming Language

Try: Lean playground

Textbooks/tutorials:

- Natural Number Game (tutorial)
- Mechanics of proof (introductory)
- Mathematics in Lean (writing math in Lean)
- Logic and proof (dependent type theory and Lean implementation)
- Theorem proving in Lean (computer-science side)

Browse: `mathlib` documentation

Search: LeanSearch

Ask: Zulip chat